The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases (Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!

FRED TALKS

23rd June 2026

CONTENTS

A non-anthropomorphized view of LLMs

HALVAR.FLAKE · 14 SEP 2025 · SOURCE

In many discussions where questions of "alignment" or "AI safety" crop up, I am baffled by seriously intelligent people imbuing almost magical human-like powers to something that - in my mind - is just MatMul with interspersed nonlinearities.

In one of these discussions, somebody correctly called me out on the simplistic nature of this argument - "a brain is just some proteins and currents". I felt like I should explain my argument a bit more, because it feels less simplistic to me:

The space of words

The tokenization and embedding step maps individual words (or tokens) to some vectors. So let us imagine for a second that we have in front of us. A piece of text is then a path through this space - going from word to word to word, tracing a (possibly convoluted) line.

Imagine now that you label each of the "words" that form the path with a number: The last word with 1, counting forward until you hit the first word or the maximum context length . If you've ever played the game "Snake", picture something similar, but played in very high-dimensional space - you're moving forward through space with the tail getting truncated off.

The LLM takes your previous path into account, calculates probabilities for the next point to go to, and then makes a random pick into the next point according to these probabilities. An LLM instantiated with a fixed random seed is a mapping of the form .

In my mind, the paths generated by these mappings look a lot like strange attractors in dynamical systems - complicated, convoluted paths that are structured-ish.

Learning the mapping

We obtain this mapping by training it to mimic human text. For this, we use approximately all human writing we can obtain, plus corpora written by human experts on a particular topic, plus some automatically generated pieces of text in domains where we can automatically generate and validate them.

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs… but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application’s state, with support for locking and write-ahead logging in case of failures; there’s even a sentence in your onboarding doc saying “NEVER TOUCH STATE IN FOOBARDB DIRECTLY!”. You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool’s existence, or maybe it just preferred FooBarDB’s in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name “delete-everything”. Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · SOURCE

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments…). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

Paths to avoid

There are certain language sequences we wish to avoid - because the sequences these models generate try to mimic human speech in all it's empirical structure, but we feel that some of the things that humans have empirically written are very undesirable to be generated. We also feel that a variety of other paths should ideally not be generated, if - when interpreted by either humans or other computer systems - undesirable results arise.

We can't specify strictly in a mathematical sense which paths we would prefer not to generate, but we can provide examples and counterexamples, and we try to hence nudge the complicated learnt distribution away from them.

"Alignment" for LLMs

Alignment and safety for LLMs mean that **we should be able to quantify and bound the probability with which certain undesirable sequences are generated.** The trouble is that we largely fail at describing "undesirable" except by example, which makes calculating bounds difficult.

For a given LLM (without random seed) and sequence, it is trivial to calculate the probability of the sequence to be generated. So if we had a way of somehow summing or integrating over these probabilities, we could say with certainty "this model will generate an undesirable sequence once every N model evaluations". We can't, currently, and that sucks, but at the heart, this is the mathematical and computational problem we'd need to solve.

The surprising utility of LLMs

LLMs solve a large number of problems that could previously not be solved algorithmically. NLP (as the field was a few years ago) has largely been solved.

I can write a request in plain English to summarize a document for me and put some key datapoints from the document in a structured JSON format, and modern models will just do that. I can ask a model to generate a children's book story involving raceboats and generate illustrations, and the model will generate something that is passable. And much more, all of which would have seemed like absolute science fiction 5-6 years ago.

We're on a pretty steep improvement curve, so I expect the number of currently-intractable problems that these models can solve to keep increasing for a while.

Where anthropomorphization loses me

The moment that people ascribe properties such as "consciousness" or "ethics" or "values" or "morals" to these learnt mappings is where I tend to get lost. We are speaking about a big recurrence equation that produces a new word, and that stops producing words if we don't crank the shaft.

To me, wondering if this contraption will "wake up" is similarly bewildering as if I was to ask a computational meteorologist if he isn't afraid of his meteorological numerical calculation will "wake up".

I am baffled that the AI discussions seem to never move away from treating a function to generate sequences of words as something that resembles a human. Statements such as "an AI agent could become an insider threat so it needs monitoring" are simultaneously unsurprising (you have a randomized sequence generator fed into your shell, literally **anything** can happen!) and baffling (you talk as if you believe the dice you play with had a mind of their own and could decide to conspire against you).

Instead of saying "we cannot ensure that no harmful sequences will be generated by our function, partially because we don't know how to specify and enumerate harmful sequences", we talk about "behaviors", "ethical constraints", and "harmful actions in pursuit of their goals". All of these are anthropocentric concepts that - in my mind - do not apply to functions or other mathematical objects. And using them muddles the discussion, and our thinking about what we're doing when we create, analyze, deploy and monitor LLMs.

This muddles the public discussion. We have many historical examples of humanity ascribing bad random events to "the wrath of god(s)" (earthquakes, famines, etc.), "evil spirits" and so forth. The fact that intelligent highly educated researchers talk about these mathematical objects in anthropomorphic terms makes the technology seem mysterious, scary, and magical.

We should think in terms of "this is a function to generate sequences" and "by providing prefixes we can steer the sequence generation around in the space of words and change the probabilities for output sequences". And for every possible undesirable output sequence of a length smaller than , we can pick a context that maximizes the probability of this undesirable output sequence.

A much clearer formulation, which helps more clearly articulate the problems to solve.

That’s too black and white. You have content creators willing to put in work. Shift their black-hat SEO over to an acceptable white-hat SEO. Document what’s acceptable. What’s considered malicious.

Focus on feedback. Make the content-optimization path rewarding. Give creators feedback+guidance during creation. LLMs help detect spam in real-time. They push content creators to author what’s useful not malicious.

Give content creators feedback. Once you’ve identified what ‘good’ is, give those content authors dopamine hits. Tell content creators what queries perform well for the content. Make it real-time, sticky, fun. Incentivize and reward behaviors you want - and punish those you don’t.

Steer content creators towards the content you want. Don’t hope algorithms can overcome drivel and slop.

Your search engine isn’t a bundle of algorithms ranking for users. It’s a relationship between authors and users. That bitter lesson puts grep above the smartest reranker and embedding model. Forget it at your peril.

Enjoy softwaredoug in training course form!

Starting May 18!

Signup here - http://maven.com/softwaredoug/cheat-at-search

I hope you join me at Cheat at Search with Agents to learn use agents in search. build better RAG and use LLMs in query understanding.

You learn that the trick to RAG isn’t creating good question answering systems. Its writing better answers.

The enterprise search morass… is over?

Compare how coding agents program us to the eternal nightmare of your company’s wiki.

When people write in the wiki, they do it for themselves. Nobody is incentivized to make information findable by anyone. No author sits obsessing how to ‘SEO’ the content for Confluence search.

If someone is ‘lucky’ enough to find your article, they can barely understand it. It’s written in your team’s jargon. And it’s probably out of date anyways.

For these reasons, I always tell people they’d do better to write a public blog article. Blogs assume zero context. They’re clearly timestamped as an artifact of a moment in time. To avoid shame or promote your career, you’re incentivized to create something useful, interesting, and findable.

Writing for a coding agent has the same property. You’re writing documentation for a targeted reader. You care to SEO your content for Claude Code + grep.

Enterprise search fails because there’s no feedback loop between authorship and findability. Coding agents solve that, finally giving some hope to organizing company information into findable knowledge.

The real outcome of coding agents isn’t just code generation. It’s closing the feedback loop between company context and the next piece of work being done. That old fable of Enterprise Search - improving workforce productivity by leveraging existing insight - can finally become true.

Feedback to content authors drives user relevance

Look around your system. Who creates the content? What’s in it for them to make content findable?

A recurring theme in my consulting looks something like this - a job search company notices spammy behavior from job posters. They keyword stuff every programming language in some hidden text element. The usual reaction: that’s spammy and we should ban them.

Why many AI luminaries tend to anthropomorphize

Perhaps I am fighting windmills, or rather a self-selection bias: A fair number of current AI luminaries have self-selected by their belief that they might be the ones getting to AGI - "creating a god" so to speak, the creation of something like life, as good as or better than humans. You are more likely to choose this career path if you believe that it is feasible, and that current approaches might get you there. Possibly I am asking people to "please let go of the belief that you based your life around" when I am asking for an end to anthropomorphization of LLMs, which won't fly.

Why I think human consciousness isn't comparable to an LLM

The following is uncomfortably philosophical, but: In my worldview, humans are dramatically different things than a function . For hundreds of millions of years, nature generated new versions, and only a small number of these versions survived. Human thought is a poorly-understood process, involving enormously many neurons, extremely high-bandwidth input, an extremely complicated cocktail of hormones, constant monitoring of energy levels, and millions of years of harsh selection pressure.

We understand essentially nothing about it. In contrast to an LLM, given a human and a sequence of words, I cannot begin putting a probability on "will this human generate this sequence".

To repeat myself: To me, considering that any human concept such as ethics, will to survive, or fear, apply to an LLM appears similarly strange as if we were discussing the feelings of a numerical meteorology simulation.

The real issues

The function class represented by modern LLMs are very useful. Even if we never get anywhere close to AGI and just deploy the current state of technology everywhere where it might be useful, we will get a dramatically different world. LLMs might end up being similarly impactful as electrification.

My grandfather lived from 1904 to 1981, a period which encompassed moving from gas lamps to electric, the replacement of horse carriages by cars, nuclear power, transistors, all the way to computers. It also spanned two world wars, the rise of Communism and Stalinism, almost the entire lifetime of the USSR and GDR etc. The world on his birth looked nothing like the world when he died.

Navigating the dramatic changes of the next few decades while trying to avoid world wars and murderous ideologies is difficult enough without muddying our thinking.

Every layer of review makes you 10x slower

AVERY PENNARUN · 26 MAR 2026 · SOURCE

2026-03-16 »

Every layer of review makes you 10x slower

We've all heard of those network effect laws: the value of a network goes up with the square of the number of members. Or the cost of communication goes up with the square of the number of members, or maybe it was n log n, or something like that, depending how you arrange the members. Anyway doubling a team doesn't double its speed; there's coordination overhead. Exactly how much overhead depends on how badly you botch the org design.

But there's one rule of thumb that someone showed me decades ago, that has stuck with me ever since, because of how annoyingly true it is. The rule is annoying because it doesn't *seem* like it should be true. There's no theoretical basis for this claim that I've ever heard. And yet, every time I look for it, there it is.

Here we go:

Every layer of approval makes a process 10x slower

I know what you're thinking. Come on, 10x? That's a lot. It's unfathomable. Surely we're exaggerating.

Nope.

Just to be clear, we're counting "wall clock time" here rather than effort. Almost all the extra time is spent sitting and waiting.

Look:

- Code a simple bug fix
  30 minutes

> What is a runner test?

Answer:

> Verifies that small-dollar deposits and withdrawals can be executed against real test bank accounts during account reconciliation.

Yes that's literally an answer to the question.

But I wouldn't consider this "good documentation". It's not anchored to any concepts from the question. It's just an out of context bit of text.

If you wrote this for a coding agent, you'd probably organize the chunk to give hints about why it's useful.

> docs/tests/runner.md
>
> # Runner Tests
>
> ## Purpose
>
> We need to ensure the system runs end to end
>
> ## Usage
>
> Use this style of test sparingly. Unit and standard e2e tests should usually be preferred.
>
> ## What is it
>
> A runner test actually interacts with real-life test bank accounts to ensure we can correctly deposit / withdraw money and that the full banking system works.

That's a far more descriptive piece of content to an LLM. Being encouraged to write *for the LLM itself* makes all the difference.

(As an aside, LLMs see candidate *answers* in their context before the user's question - so just treating RAG as question answering doesn't work).

What often matters isn't algorithms. It's programming the content author to create useful LLM context. That moves content discovery from random, disconnected factoids to documentation connecting information to problems to solve.

Search has its own bitter lesson

DOUG TURNBULL · 22 JUN 2026 · SOURCE

An agent with grep does quite well at search.

It’s shocking to search technologists. We want to cocoon the problem in technology. We care about algorithms, knowledge graphs, ranking: ever-and-ever smarter retrieval.

It turns out, though, you can play a bit of a trick. If you convince everyone to optimize content for your search engine, you’ll have built the best search engine.

There’s Sutton’s bitter lesson about unleashing raw compute on a problem. But there’s a different bitter lesson in search: algorithms matter less than convincing the world to optimize for your search engine.

Sure, technology acts as the trigger. In Web Search, Google shifted away from text relevance to prioritize authoritativeness (PageRank). Their success told the market: create content others want to link to. They reshaped the Web for good (or ill) around that incentive.

Google crowdsourced search quality to the Web. Claude Code outsources it to every developer.

Developers depend on agents to find documentation on the file system. We like that dopamine hit. That thrill of Claude Code knowing exactly how to reason about our code.

When agents make boneheaded mistakes, we stress. We frantically update documentation, we link to it from AGENTS.md, we add a new document, we place it in a useful location. We do anything - absolutely anything - to ensure agents know how to find and understand documentation.

Just as the SEO expert reads the tea leaves of Google’s algorithm. The software developer writes to the LLM to make documentation findable + useful.

A search system with these incentives overcomes technologically sophisticated systems. For example, classic RAG rests on a foundation of technically sophisticated question answering. Answers often look like conversationally valid responses, but with little linkage back to questions it might anser:

Question:

- Get it code reviewed by the peer next to you
  300 minutes → 5 hours → half a day

- Get a design doc approved by your architects team first
  50 hours → about a week

- Get it on some other team’s calendar to do all that
  (for example, if a customer requests a feature)
  500 hours → 12 weeks → one fiscal quarter

I wish I could tell you that the next step up — 10 quarters or about 2.5 years — was too crazy to contemplate, but no. That’s the life of an executive sitting above a medium-sized team; I bump into it all the time even at a relatively small company like Tailscale if I want to change product direction. (And execs sitting above large teams can’t actually do work of their own at all. That's another story.)

AI can’t fix this

First of all, this isn’t a post about AI, because AI’s direct impact on this problem is minimal. Okay, so Claude can code it in 3 minutes instead of 30? That’s super, Claude, great work.

Now you either get to spend 27 minutes reviewing the code yourself in a back-and-forth loop with the AI (this is actually kinda fun); or you save 27 minutes and submit unverified code to the code reviewer, who will still take 5 hours like before, but who will now be mad that you’re making them read the slop that you were too lazy to read yourself. Little of value was gained.

Now now, you say, that’s not the value of agentic coding. You don’t use an agent on a 30-minute fix. You use it on a monstrosity week-long project that you and Claude can now do in a couple of hours! Now we’re talking. Except no, because the monstrosity is so big that your reviewer will be extra mad that you didn’t read it yourself, and it’s too big to review in one chunk so you have to slice it into new bite-sized chunks, each with a 5-hour review cycle. And there’s no design doc so there’s no intentional architecture, so eventually someone’s going to push back on that and here we go with the design doc review meeting, and now your monstrosity week-long project that you did in two hours is... oh. A week, again.

I guess I could have called this post Systems Design 4 (or 5, or whatever I’m up to now, who knows, I’m writing this on a plane with no wifi) because yeah, you guessed it. It's Systems Design time again.

The only way to sustainably go faster is fewer reviews

It’s funny, everyone has been predicting the Singularity for decades now. The premise is we build systems that are so smart that they themselves can build the next system that is even smarter, that builds the next smarter one, and so on, and once we get that started, if they keep getting smarter faster enough, then the incremental time (t) to achieve a unit (u) of improvement goes to zero, so (u/t) goes to infinity and foom.

Anyway, I have never believed in this theory for the simple reason we outlined above: the majority of time needed to get anything done is not actually the time doing it. It’s wall clock time. Waiting. Latency.

And you can’t overcome latency with brute force.

I know you want to. I know many of you now work at companies where the business model kinda depends on doing exactly that.

Sorry.

But you can’t just *not review* things!

Ah, well, no, actually yeah. You really can’t.

There are now many people who have seen the symptom: the start of the pipeline (AI generated code) is so much faster, but all the subsequent stages (reviews) are too slow! And so they intuit the obvious solution: stop reviewing then!

The result might be slop, but if the slop is 100x cheaper, then it only needs to deliver 1% of the value per unit and it's still a fair trade. And if your value per unit is even a mere 2% of what it used to be, you’ve doubled your returns! Amazing.

There are some pretty dumb assumptions underlying that theory; you can imagine them for yourself. Suffice it to say that this produces what I will call the AI Developer’s Descent Into Madness:

1. Whoa, I produced this prototype so fast! I have super powers!
2. This prototype is getting buggy. I’ll tell the AI to fix the bugs.
3. Hmm, every change now causes as many new bugs as it fixes.
4. Aha! But if I have an AI agent also review the code, it can find its own bugs!
5. Wait, why am I personally passing data back and forth between agents

But, it’s not just about company size. I think engineering teams at any company can get smaller, and have better defined interfaces between them.

Maybe you could have multiple teams inside a company competing to deliver the same component. Each one is just a few people and a few coding bots. Try it 100 ways and see who comes up with the best one. Again, quality by evolution. Code is cheap but good ideas are not. But now you can try out new ideas faster than ever.

Maybe we’ll see a new optimal point on the monoliths-microservices continuum. Microservices got a bad name because they were too micro; in the original terminology, a “micro” service was exactly the right size for a “two pizza team” to build and operate on their own. With AI, maybe it's one pizza and some tokens.

What’s fun is you can also use this new, faster coding to experiment with different module *boundaries* faster. *Features* are still hard for lots of reasons, but refactoring and automated integration testing are things the AIs excel at. Try splitting out a module you were afraid to split out before. Maybe it'll add some lines of code. But suddenly lines of code are cheap, compared to the coordination overhead of a bigger team maintaining both parts.

Every team has some monoliths that are a little too big, and too many layers of reviews. Maybe we won't get all the way to Singularity. But, we can engineer a much better world. Our problems are solvable.

It just takes trust.

Related

Highlights on "quality," and Deming's work as it applies to software development (2016)

Systems design 3: LLMs and the semantic revolution (2025)

Unrelated

SimSWE part 2: The perils of multitasking (2017)

I think we're going to be stuck with these systems pipeline problems for a long time. Review pipelines — layers of QA — don't work. Instead, they make you slower while hiding root causes. Hiding causes makes them harder to fix.

But, the call of AI coding is strong. That first, fast step in the pipeline is *so fast!* It really does feel like having super powers. I want more super powers. What are we going to do about it?

Maybe we finally have a compelling enough excuse to fix the 20 years of problems hidden by code review culture, and replace it with a real culture of quality.

I think the optimists have half of the right idea. Reducing review stages, even to an uncomfortable degree, is going to be needed. But you can't just reduce review stages without something to replace them. That way lies the Ford Pinto or any recent Boeing aircraft.

The complete package, the table flip, was what Deming brought to manufacturing. You can't half-adopt a "total quality" system. You need to eliminate the reviews *and* obsolete them, in one step.

How? You can fully adopt the new system, in small bites. What if some components of your system can be built the new way? Imagine an old-school U.S. auto manufacturer buying parts from Japanese suppliers; wow, these parts are so well made! Now I can start removing QA steps elsewhere because I can just assume the parts are going to work, and my job of "assemble a bigger widget from the parts" has a ton of its complexity removed.

I like this view. I've always liked small beautiful things, that's my own bias. But, you can assemble big beautiful things from small beautiful things.

It's a lot easier to build those individual beautiful things in small teams that trust each other, that know what quality looks like *to them*. They deliver their things to customer teams who can clearly explain what quality looks like *to them*. And on we go. Quality starts bottom-up, and spreads.

I think small startups are going to do really well in this new world, probably better than ever. Startups already have fewer layers of review just because they have fewer people. Some startups will figure out how to produce high quality components quickly; others won't and will fail. Quality by natural selection?

Bigger companies are gonna have a harder time, because their slow review systems are baked in, and deleting them would cause complete chaos.

6. I need an agent framework
7. I can have my agent write an agent framework!
8. Return to step 1

It's actually alarming how many friends and respected peers I've lost to this cycle already. Claude Code only got good maybe a few months ago, so this only recenlty started happening, so I assume they will emerge from the spiral eventually. I mean, I hope they will. We have no way of knowing.

Why we review

Anyway we know our symptom: the pipeline gets jammed up because of too much new code spewed into it at step 1. But what's the root cause of the clog? Why doesn't the pipeline go faster?

I said above that this isn't an article about AI. Clearly I'm failing at that so far, but let's bring it back to humans. It goes back to the annoyingly true observation I started with: every layer of review is 10x slower. As a society, we know this. Maybe you haven't seen it before now. But trust me: people who do org design for a living know that layers are expensive... and they still do it.

As companies grow, they all end up with more and more layers of collaboration, review, and management. Why? Because otherwise mistakes get made, and mistakes are increasingly expensive at scale. The average value added by a new feature eventually becomes lower than the average value lost through the new bugs it causes. So, lacking a way to make features produce more value (wouldn't that be nice!), we try to at least reduce the damage.

The more checks and controls we put in place, the slower we go, but the more monotonically the quality increases. And isn't that the basis of *continuous improvement?*

Well, sort of. Monotonically increasing quality is on the right track. But "more checks and controls" went off the rails. That's *only one way* to improve quality, and it's a fraught one.

"Quality Assurance" reduces quality

I wrote a few years ago about W. E. Deming and the "new" philosophy around quality that he popularized in Japanese auto manufacturing. (Eventually U.S. auto manufacturers more or less got the idea. So far the software industry hasn't.)

One of the effects he highlighted was the problem of a “QA” pass in a factory: build widgets, have an inspection/QA phase, reject widgets that fail QA. Of course, your inspectors probably miss some of the failures, so when in doubt, add a second QA phase after the first to catch the remaining ones, and so on.

In a simplistic mathematical model this seems to make sense. (For example, if every QA pass catches 90% of defects, then after two QA passes you’ve reduced the number of defects by 100x. How awesome is that?)

But in the reality of agentic humans, it’s not so simple. First of all, the incentives get weird. The second QA team basically serves to evaluate how well the first QA team is doing; if the first QA team keeps missing defects, fire them. Now, that second QA team has little incentive to produce that outcome for their friends. So maybe they don’t look too hard; after all, the first QA team missed the defect, it’s not unreasonable that we might miss it too.

Furthermore, the first QA team knows there is a second QA team to catch any defects; if I don’t work too hard today, surely the second team will pick up the slack. That's why they're there!

Also, the team making the widgets in the first place doesn’t check their work too carefully; that’s what the QA team is for! Why would I slow down the production of every widget by being careful, at a cost of say 20% more time, when there are only 10 defects in 100 and I can just eliminate them at the next step for only a 10% waste overhead? It only makes sense. Plus they'll fire me if I go 20% slower.

To say nothing of a whole engineering redesign to improve quality, that would be super expensive and we could be designing all new widgets instead.

Sound like any engineering departments you know?

Well, this isn’t the right time to rehash Deming, but suffice it to say, he was on to something. And his techniques worked. You get things like the famous Toyota Production System where they eliminated the QA phase entirely, but gave everybody an “oh crap, stop the line, I found a defect!” button.

Famously, US auto manufacturers tried to adopt the same system by installing the same “stop the line” buttons. Of course, nobody pushed those buttons. They were afraid of getting fired.

Trust

The basis of the Japanese system that worked, and the missing part of the American system that didn’t, is trust. Trust among individuals that your boss Really Truly Actually wants to know about every defect, and wants you to stop the line when you find one. Trust among managers that executives were serious about quality. Trust among executives that individuals, given a system that can work and has the right incentives, will produce quality work and spot their own defects, and push the stop button when they need to push it.

But, one more thing: trust that the system *actually does work*. So first you need a system that will work.

Fallibility

AI coders are fallible; they write bad code, often. In this way, they are just like human programmers.

Deming’s approach to manufacturing didn’t have any magic bullets. Alas, you can’t just follow his ten-step process and immediately get higher quality engineering. The secret is, you have to get your engineers to engineer higher quality into the whole system, from top to bottom, repeatedly. Continuously.

Every time something goes wrong, you have to ask, “How did this happen?” and then do a whole post-mortem and the Five Whys (or however many Whys are in fashion nowadays) and fix the underlying Root Causes so that it doesn’t happen again. “The coder did it wrong” is never a root cause, only a symptom. Why was it possible for the coder to get it wrong?

The job of a code reviewer isn't to review code. It's to figure out how to obsolete their code review comment, that whole class of comment, in all future cases, until you don't need their reviews at all anymore.

(Think of the people who first created "go fmt" and how many stupid code review comments about whitespace are gone forever. Now that's engineering.)

By the time your review catches a mistake, the mistake has already been made. The root cause happened already. You're too late.

Modularity

I wish I could tell you I had all the answers. Actually I don’t have much. If I did, I’d be first in line for the Singularity because it sounds kind of awesome.